Infinity OS
Cognitive Invariance Layer v1.0
Engineering Specification

Implementation-Grade Engineering Reference — CIL + IIE Conformant Systems

Document Type

Engineering Specification (ES)

System

Infinity OS / Cognitive Invariance Layer + Invariant Inference Engine

Version

1.0

Status

Draft-Stable

Date

June 2026

Companion Document

Infinity OS — CIL v1.0 Formal Architecture Specification

Prepared By

Infinity OS Platform Engineering

Classification

Internal — Engineering Reference

Table of Contents

---

1. Scope & Audience

1.1 Purpose

This document provides the complete engineering-level specification for building a **CIL+IIE**-conformant runtime under **Infinity OS**. It is the implementation companion to the Infinity OS CIL v1.0 Formal Architecture Specification and covers all data structures, serialization schemas, API surfaces, state machines, and implementation flows. Engineers MUST implement every interface and schema defined here to achieve conformance. Where the Formal Architecture Specification defines *what* the system does, this document specifies *how* to build it.

> ⓘ **SCOPE NOTE**
>
> This specification is authoritative for all CIL+IIE implementation work. In any conflict between this document and informal guidance, this document takes precedence. Conflicts with the Formal Architecture Specification must be escalated to the Platform Engineering lead for resolution.

1.2 Audience

- **Systems engineers** implementing **CL**, **MSL**, or **SL** components
- **Platform engineers** building or integrating the **IIE** (Invariant Inference Engine)

- **Governance engineers** implementing the **GovernanceService**
- **QA engineers** building conformance test suites against the checklist in Section 10

Readers are assumed to have working knowledge of distributed systems, RPC frameworks (gRPC / Cap'n Proto), and the Infinity OS Formal Architecture Specification.

1.3 Normative References

Reference	Title / Description
`[CIL-FAS]`	Infinity OS — CIL v1.0 Formal Architecture Specification (companion document)
`[NIST-800-208]`	NIST SP 800-208: Recommendation for Stateful Hash-Based Signature Schemes
`[RFC-3339]`	RFC 3339: Date and Time on the Internet — Timestamps
`[JSON-07]`	JSON Schema Draft-07 (http://json-schema.org/draft-07/schema)
`[PROTO3]`	Protocol Buffers Language Guide v3 (proto3)
`[CAPNP-1]`	Cap'n Proto Schema Language v1.0 (capnproto.org)
`[WASM-2]`	WebAssembly Core Specification 2.0 (W3C)

1.4 Notation Conventions

Keyword	Meaning
MUST / **MUST NOT**	Normative requirement. Violation constitutes non-conformance and MUST be reported as a Violation object.
SHOULD / **SHOULD NOT**	Strong recommendation. Deviation requires documented justification.
MAY	Optional. Implementations are free to include or omit.

Keyword	Meaning
`monospaced`	Type names, field names, identifiers, schema values, and code references.
Bold	First use of a defined term, interface name, or type name.
Italic	Optional fields in schema tables; conceptual terms.

---

2. Core Data Structures

Four canonical data structures form the complete information model of the CIL. Every interface, API, and serialization format in this specification derives from these definitions. Implementations MUST use these structures without modification, extension, or elision of required fields.

2.1 Epoch

The **Epoch** is the atomic unit of CL state transition. Every CL state change MUST be represented as an Epoch. No ad-hoc mutation of CL state is permitted outside of a typed, IIE-authorized Epoch.

```
{   "epoch_id":       "EPOCH-UUID",   "parent_epoch":   "EPOCH-UUID | null",
"timestamp":      "RFC3339",   "type":           "CREATE | MUTATE | ROLLBACK
| GOVERNANCE | CHECKPOINT",   "intent":         "string",   "input":
{ "any": "structured input to CL" },   "pre_state_hash": "SHA256",
"post_state_hash":"SHA256 | null",   "diff":           { "any": "state delta"
},   "governance_context": {     "proposal_id":  "string | null",
"law_context":  ["LAW-ID-1", "LAW-ID-2"]   },   "audit": {     "signature":
"SIG",     "validator_id": "IIE-ID"   } }
```

2.1.1 Field Specifications

Field	Type	Required	Description
`epoch_id`	string (UUID)	**YES**	Globally unique epoch identifier. MUST conform to UUID v4 format.
`parent_epoch`	string | null	NO	Parent epoch ID. MUST be null for

Field	Type	Required	Description
			genesis epochs. MUST reference a COMMITTED epoch for all non-genesis epochs.
`timestamp`	string (RFC3339)	**YES**	Creation timestamp. MUST conform to RFC 3339. MUST be UTC or include explicit timezone offset.
`type`	enum	**YES**	Epoch type. One of: `CREATE`, `MUTATE`, `ROLLBACK`, `GOVERNANCE`, `CHECKPOINT`.
intent	string	NO	Human-readable description of the epoch's purpose. Stored in audit ledger. SHOULD be populated for all non-programmatic epochs.
input	object | null	NO	Structured input to CL for this epoch. Schema is CL-implementation-specific but MUST be JSON-serializable.
`pre_state_hash`	string (SHA-256)	**YES**	Hash of CL state immediately prior to this epoch. MUST be computed using SHA-256 over the canonical JSON serialization of the pre-state.
`post_state_hash`	string (SHA-256) | null	**YES**	Hash of CL state after commit. MUST be null until the epoch reaches COMMITTED state.

Field	Type	Required	Description
			Written by IIE at CommitEpoch time.
diff	object | null	NO	Deterministic state delta. MUST be replayable — given `pre_state_hash` and `diff`, replay MUST produce a state whose hash equals `post_state_hash`.
`governance_context.proposal_id`	string | null	**YES**	Bound governance proposal ID. MUST be null if no active governance proposal applies to this epoch.
`governance_context.law_context`	string[]	**YES**	Array of LAW-IDs applicable to this epoch. MAY be empty array if no laws apply.
`audit.signature`	string	**YES**	Cryptographic signature over the canonical serialization of the epoch (excluding the `audit` field itself). Algorithm: per [NIST-800-208].
`audit.validator_id`	string	**YES**	Identity of the IIE instance that authorized this epoch. MUST be an IIE instance that is registered in the platform trust registry.

2.1.2 Epoch Type Semantics

Type	Meaning	Governance Authorization Required
`CREATE`	Initializes a new CL state tree. Establishes genesis `pre_state_hash`.	NO (unless a law-constraint exists on the target CL)
`MUTATE`	Modifies existing CL state. Most common epoch type in steady-state operation.	CONDITIONAL — required if any applicable law mandates pre-authorization
`ROLLBACK`	Reverts CL to a prior CHECKPOINT epoch. MUST reference the target checkpoint in `parent_epoch`.	**YES** — always requires GovernanceTicket with ROLLBACK decision
`GOVERNANCE`	Epoch carrying a governance decision payload. Written by IIE on receipt of a DECIDED GovernanceTicket.	**YES** — inherently governance-originated
`CHECKPOINT`	Persistent snapshot of CL state. Used as rollback target. MUST be immutable after COMMITTED.	NO

⚠️ **IMPLEMENTATION CONSTRAINT**

post_state_hash

MUST be null until the epoch reaches COMMITTED state. Any epoch submitted to

IIE with a non-null

post_state_hash

in PROPOSED state MUST be rejected with a FATAL Violation. The

diff

field MUST be deterministic — given identical

pre_state_hash

and

input

, replay MUST produce the identical

post_state_hash

. Non-deterministic diffs constitute an INV-1 violation.

2.2 Invariant

An **Invariant** is a machine-verifiable rule enforced by the IIE at pre-execution, runtime, or post-execution checkpoints. Invariants are organized into a three-level hierarchy: Constitutional (L0), Statutory (L1), and Policy (L2). Higher-level invariants require governance authorization to modify or disable.

```
{   "invariant_id": "INV-001",   "level":        "CONSTITUTIONAL | STATUTORY
| POLICY",   "description":  "string",   "scope":        ["CL", "MSL", "SL",
"GOVERNANCE"],   "check": {     "type":       "PRE | POST | RUNTIME | ALL",
"logic":      "wasm://invariant-validators/inv-001.wasm"   },   "severity":
"FATAL | ERROR | WARNING",   "on_violation": {     "action":           "HALT
| BLOCK | ESCALATE | LOG",     "governance_route":  true   } }
```

2.2.1 Field Specifications

Field	Type	Required	Description
`invariant_id`	string	**YES**	Canonical ID. Format: `INV-###` (zero-padded, three digits minimum). MUST be globally unique within the IIE invariant registry.
`level`	enum	**YES**	`CONSTITUTIONAL` (L0): immutable except by platform reset. `STATUTORY` (L1): amendable via governance.

Field	Type	Required	Description
			`POLICY` (L2): configurable by authorized operators.
`description`	string	**YES**	Human-readable invariant statement. MUST be precise enough to serve as a test assertion.
`scope`	string[]	**YES**	Layers where this invariant applies. Valid values: `CL`, `MSL`, `SL`, `GOVERNANCE`. At least one value MUST be present.
`check.type`	enum	**YES**	When IIE evaluates: `PRE` (before execution), `POST` (after execution), `RUNTIME` (during execution via RG), `ALL` (at all three points).
`check.logic`	string	**YES**	Reference to the validator. See Logic Reference Formats below.
`severity`	enum	**YES**	`FATAL`: IIE halts and escalates immediately. `ERROR`: IIE blocks and escalates to governance. `WARNING`: IIE logs but permits continuation.
`on_violation.action`	enum	**YES**	`HALT`: terminate execution. `BLOCK`: reject epoch and block. `ESCALATE`: raise GovernanceTicket. `LOG`: record in

Field	Type	Required	Description
			audit ledger only.
`on_violation.governance_route`	bool	**YES**	If `true`, IIE MUST submit a GovernanceTicket via GB on any violation of this invariant.

2.2.2 Logic Reference Formats

Format	Example	Description
WASM module	`wasm://invariant-validators/inv-001.wasm`	Reference to a WASM module implementing the `validate(epoch) → bool` export. Preferred format for complex invariants.
Function hash	`sha256://abc123...#validate_determinism`	Reference to a function by its content-addressed hash. IIE resolves from the invariant artifact store.
Embedded expression	`state.pre_hash == compute_hash(state.input)`	Inline expression in the CIL rule DSL. Permitted for simple, side-effect-free assertions only.

2.3 PhysicsThreat

A **PhysicsThreat** is the normalized representation of a substrate-level anomaly. PhysicsThreats ALWAYS originate in the SL as raw **PhysicsEvent** objects, are normalized by the MSL, and are delivered to the IIE's **DTD** (Drift & Threat Detector) subsystem. The MSL MUST NOT pass raw, unnormalized PhysicsEvents to the IIE.

```
{   "threat_id":   "PTA-UUID",   "timestamp":   "RFC3339",   "substrate":
"CLASSICAL | QUANTUM | LATTICE | HYBRID",   "category":
"COHERENCE_COLLAPSE | ENTANGLEMENT_POISONING | LATTICE_DRIFT
| TIMING_ANOMALY | UNKNOWN",   "severity":    "INFO | WARNING | CRITICAL",
"metrics": {     "raw":        { "any": "substrate-specific telemetry" },
"normalized": { "any": "normalized values (substrate-agnostic units)" }   },
"msl_context": {     "substrate_profile": "SAI-PROFILE-ID",
"capabilities":      ["CAP-IDs"]   },   "governance_required":  true,
"recommended_action":   "string" }
```

2.3.1 Threat Category Definitions

Category	Applicable Substrate	Description
`COHERENCE_COLLAPSE`	QUANTUM	Qubit coherence time exceeded operational bounds; quantum state integrity cannot be guaranteed. MUST trigger CRITICAL severity if coherence window < minimum threshold.
`ENTANGLEMENT_POISONING`	QUANTUM	Entanglement relationship corrupted by environmental decoherence or measurement interference. Any entangled state used in an in-flight epoch MUST be invalidated.
`LATTICE_DRIFT`	LATTICE	Lattice parameter drift outside operational bounds specified in the SubstrateProfile. May indicate hardware degradation or environmental interference.
`TIMING_ANOMALY`	CLASSICAL, HYBRID	Execution timing outside the determinism contract bounds (e.g., timeout_ms exceeded or clock skew detected). Implicates determinism guarantee.
`UNKNOWN`	ANY	Anomaly detected but category cannot be determined by MSL normalization logic. IIE MUST treat UNKNOWN as WARNING severity minimum.

⚠️ **NORMALIZATION REQUIREMENT**

metrics.raw

MUST contain the unmodified substrate-specific telemetry payload as received from SL.

metrics.normalized

MUST use a common, substrate-agnostic schema that does NOT expose substrate-specific units, qubit indices, or hardware identifiers to IIE logic. This isolation MUST be enforced — IIE MUST NOT branch on substrate identity from

metrics.raw

.

2.4 Violation

A **Violation** is the canonical error object for any breach of an invariant, law, or governance constraint detected by the IIE. All IIE API error returns are typed as Violations. Violations are append-only entries in the audit ledger and MUST NOT be modified post-write.

```
{   "violation_id":  "VIO-UUID",   "timestamp":     "RFC3339",   "source":
"CL | MSL | SL | GOVERNANCE | IIE",   "invariant_id":  "INV-### | null",
"law_id":        "LAW-### | null",   "epoch_id":      "EPOCH-UUID | null",
"severity":      "FATAL | ERROR | WARNING",   "details": {     "message":
"string",     "context":     { "any": "structured context" }   },
"ii_engine_action":   "HALT | BLOCK | ESCALATE | LOG",   "governance_event":
{     "submitted":    true,     "proposal_id":  "string | null"   } }
```

2.4.1 Field Specifications

Field	Type	Required	Description
`violation_id`	string (UUID)	**YES**	Globally unique violation identifier. MUST conform to UUID v4.
`timestamp`	string (RFC3339)	**YES**	Timestamp of violation detection by IIE. MUST be UTC or include

Field	Type	Required	Description
			explicit offset.
`source`	enum	**YES**	Layer where the violation originated: `CL`, `MSL`, `SL`, `GOVERNANCE`, or `IIE`.
invariant_id	string | null	NO	The INV-### that was violated. Null if the violation is law-level rather than invariant-level.
law_id	string | null	NO	The LAW-### that was violated. Null if invariant-level. At least one of `invariant_id` or `law_id` MUST be non-null.
epoch_id	string | null	NO	The epoch in context when the violation was detected. Null if violation is not epoch-bound (e.g., system-level physics threat).
`severity`	enum	**YES**	MUST match the severity of the violated invariant. May be escalated by IIE if context warrants.
`details.message`	string	**YES**	Human-readable violation message. MUST identify the specific condition that triggered the violation.
details.context	object	NO	Structured context for debugging. SHOULD include relevant field values from the epoch, invariant, or threat that

Field	Type	Required	Description
			triggered the violation.
`ii_engine_action`	enum	**YES**	Action taken by IIE in response: `HALT`, `BLOCK`, `ESCALATE`, or `LOG`.
`governance_event.submitted`	bool	**YES**	Whether IIE submitted a GovernanceTicket for this violation.
governance_event.proposal_id	string | null	NO	GovernanceTicket ID if submitted. MUST be non-null if `submitted = true`.

2.5 Interlocking Relationships

The four canonical types form a closed information model. The following formal relationships MUST hold in any conformant implementation:

Relationship	Direction	Formal Rule
Epoch → Invariant	Each Epoch is validated by one or more Invariants	IIE MUST evaluate ALL active Invariants whose `scope` includes the originating layer and whose `check.type` applies to the current lifecycle phase
Invariant → Violation	A failed Invariant check produces a Violation	The Violation's `invariant_id` MUST reference the Invariant that failed; `severity` MUST match the Invariant's `severity`
PhysicsThreat → Epoch	A CRITICAL PhysicsThreat may suspend in-flight Epochs	On CRITICAL PhysicsThreat, IIE MUST suspend all EXECUTING epochs on the affected substrate and block new ProposeEpoch calls until resolved

Relationship	Direction	Formal Rule
Violation → GovernanceTicket	A Violation with governance_route=true produces a GovernanceTicket	IIE MUST call GB.SubmitTicket within the same execution context as the Violation detection — not deferred
GovernanceTicket → Epoch	A Governance decision may roll back, resume, or terminate an Epoch	IIE MUST apply the governance decision to the referenced epoch before resuming any further epoch processing on the affected CL

---

3. Serialization Formats

All three serialization formats below are normative. Implementations MUST support JSON Schema validation for all inter-component message exchange. Implementations SHOULD support proto3 for high-throughput internal paths. Cap'n Proto is provided as the zero-copy alternative for latency-critical paths (e.g., DTD hot path).

3.1 JSON Schema (Draft-07)

The following schemas MUST be used for runtime validation of all inter-component message exchanges. The `$id` URIs are the canonical schema identifiers; implementations MUST register these URIs in their schema registry.

3.1.1 epoch.json

```
{   "$id": "https://infinity.os/schema/epoch.json",   "$schema":
"http://json-schema.org/draft-07/schema#",   "type": "object",   "required":
[     "epoch_id", "timestamp", "type",     "pre_state_hash",
"governance_context", "audit"   ],   "additionalProperties": false,
"properties": {     "epoch_id": {       "type": "string",
"description": "UUID v4 epoch identifier"     },     "parent_epoch":
{       "type": ["string", "null"],       "description": "Parent epoch UUID;
null for genesis epochs"     },     "timestamp": {       "type": "string",
"format": "date-time",       "description": "RFC 3339 creation timestamp"
},     "type": {       "type": "string",       "enum": ["CREATE", "MUTATE",
"ROLLBACK", "GOVERNANCE", "CHECKPOINT"]     },     "intent": {       "type":
"string"     },     "input": {       "type": ["object", "null"],
"additionalProperties": true     },     "pre_state_hash": {       "type":
"string",       "description": "SHA-256 hex-encoded hash of pre-execution CL
```

```
state"     },     "post_state_hash": {       "type": ["string", "null"],
"description": "SHA-256 hex-encoded hash of post-execution CL state; null
until COMMITTED"     },     "diff": {       "type": ["object", "null"],
"additionalProperties": true,       "description": "Deterministic, replayable
state delta"     },     "governance_context": {       "type": "object",
"required": ["proposal_id", "law_context"],       "additionalProperties":
false,       "properties": {         "proposal_id": { "type": ["string",
"null"] },         "law_context": {           "type": "array",
"items": { "type": "string" },           "description": "Array of applicable
LAW-IDs"         }       }     },     "audit": {       "type": "object",
"required": ["signature", "validator_id"],       "additionalProperties":
false,       "properties": {         "signature": {           "type":
"string",           "description": "Cryptographic signature per NIST SP 800-
208"         },         "validator_id": {           "type": "string",
"description": "IIE instance ID from platform trust registry"         }
}     }   } }
```

3.1.2 physics_threat.json

```
{   "$id": "https://infinity.os/schema/physics_threat.json",   "$schema":
"http://json-schema.org/draft-07/schema#",   "type": "object",   "required":
[     "threat_id", "timestamp", "substrate", "category",     "severity",
"msl_context", "governance_required"   ],   "additionalProperties": false,
"properties": {     "threat_id": {       "type": "string",
"description": "UUID v4 threat identifier (prefix: PTA-)"     },
"timestamp": { "type": "string", "format": "date-time" },     "substrate": {
"type": "string",       "enum": ["CLASSICAL", "QUANTUM", "LATTICE", "HYBRID"]
},     "category": {       "type": "string",       "enum":
[         "COHERENCE_COLLAPSE", "ENTANGLEMENT_POISONING",
"LATTICE_DRIFT", "TIMING_ANOMALY", "UNKNOWN"       ]     },     "severity": {
"type": "string",       "enum": ["INFO", "WARNING", "CRITICAL"]     },
"metrics": {       "type": "object",       "additionalProperties": false,
"properties": {         "raw": {           "type": "object",
"additionalProperties": true,           "description": "Unmodified substrate-
specific telemetry from SL"         },         "normalized":
{           "type": "object",           "additionalProperties": true,
"description": "Substrate-agnostic normalized metrics for IIE consumption"
}       }     },     "msl_context": {       "type": "object",
"required": ["substrate_profile"],       "additionalProperties": false,
"properties": {         "substrate_profile": { "type": "string" },
"capabilities": {           "type": "array",           "items": { "type":
"string" }         }       }     },     "governance_required":
{       "type": "boolean"     },     "recommended_action": {       "type":
"string"     }   } }
```

3.1.3 violation.json

```
{   "$id": "https://infinity.os/schema/violation.json",   "$schema":
"http://json-schema.org/draft-07/schema#",   "type": "object",   "required":
[     "violation_id", "timestamp", "source", "severity",     "details",
"ii_engine_action", "governance_event"   ],   "additionalProperties": false,
"properties": {     "violation_id": { "type": "string" },     "timestamp":
{ "type": "string", "format": "date-time" },     "source": {       "type":
"string",       "enum": ["CL", "MSL", "SL", "GOVERNANCE", "IIE"]     },
"invariant_id": { "type": ["string", "null"] },     "law_id": { "type":
["string", "null"] },     "epoch_id": { "type": ["string", "null"] },
"severity": {       "type": "string",       "enum": ["FATAL", "ERROR",
"WARNING"]     },     "details": {       "type": "object",       "required":
["message"],       "additionalProperties": false,       "properties": {
"message": { "type": "string" },         "context": {           "type":
```

```
"object",           "additionalProperties": true         }       }     },
"ii_engine_action": {       "type": "string",       "enum": ["HALT", "BLOCK",
"ESCALATE", "LOG"]     },     "governance_event": {       "type": "object",
"required": ["submitted"],       "additionalProperties": false,
"properties": {         "submitted": { "type": "boolean" },
"proposal_id": { "type": ["string", "null"] }       }     }   } }
```

3.2 Protocol Buffers (proto3)

The following `.proto` files are the normative proto3 definitions. All field numbers are stable and MUST NOT be changed between versions. Deprecated fields MUST be retained with their original field numbers and marked `reserved`.

3.2.1 epoch.proto

```
syntax = "proto3"; package infinity;  option go_package =
"github.com/infinity-os/cil/proto/infinity"; option java_package =
"os.infinity.cil.proto";  // Epoch is the atomic unit of CL state transition.
// Every CL state change MUST be represented as an Epoch. message Epoch
{   string epoch_id         = 1;  // UUID v4   string parent_epoch     =
2;  // Empty string for genesis epochs   string timestamp        = 3;  // RFC
3339 format   EpochType type          = 4;   string intent           = 5;
bytes  input            = 6;  // JSON-encoded structured input   string
pre_state_hash   = 7;  // SHA-256 hex   string post_state_hash  = 8;  // SHA-
256 hex; empty until COMMITTED   bytes  diff             = 9;  // JSON-
encoded deterministic delta   GovernanceContext governance_context = 10;
AuditInfo audit         = 11;    enum EpochType {     CREATE     = 0;
MUTATE     = 1;     ROLLBACK   = 2;     GOVERNANCE = 3;     CHECKPOINT = 4;
} }  message GovernanceContext {   string proposal_id         = 1;  // Empty
if no active proposal   repeated string law_context = 2; // Applicable LAW-
IDs }  message AuditInfo {   string signature   = 1;  // Cryptographic
signature per NIST SP 800-208   string validator_id = 2; // IIE instance ID }
```

3.2.2 invariant.proto

```
syntax = "proto3"; package infinity;  option go_package =
"github.com/infinity-os/cil/proto/infinity";  // Invariant is a machine-
verifiable rule enforced by IIE. message Invariant {   string invariant_id =
1;  // Format: INV-### (zero-padded)   Level  level        = 2;   string
description  = 3;   repeated string scope = 4; // CL, MSL, SL, GOVERNANCE
CheckType check_type = 5;   string    logic_ref  = 6;  // WASM URI, sha256
hash ref, or embedded expr    Severity severity           = 7;   string
on_violation_action = 8; // HALT | BLOCK | ESCALATE | LOG   bool
governance_route   = 9;    enum Level {     CONSTITUTIONAL = 0;     STATUTORY
= 1;     POLICY         = 2;   }    enum CheckType {     PRE     = 0;
POST    = 1;     RUNTIME = 2;     ALL     = 3;   }    enum Severity
{     FATAL   = 0;     ERROR   = 1;     WARNING = 2;   } }
```

3.2.3 physics_threat.proto

```
syntax = "proto3"; package infinity.physics;  option go_package =
"github.com/infinity-os/cil/proto/infinity/physics";  // PhysicsThreat is a
normalized substrate-level anomaly. // ALWAYS produced by MSL normalization
of a raw SL PhysicsEvent. message PhysicsThreat {   string threat_id
= 1;  // UUID v4, prefix PTA-   string timestamp          = 2;  // RFC 3339
string substrate          = 3;  // CLASSICAL | QUANTUM | LATTICE | HYBRID
string category           = 4;  // See category enum in engineering spec
```

```
string severity            = 5;  // INFO | WARNING | CRITICAL   bytes
metrics_raw         = 6;  // JSON-encoded substrate-specific telemetry
bytes  metrics_normalized  = 7;  // JSON-encoded substrate-agnostic metrics
string substrate_profile   = 8;  // SAI-PROFILE-ID from SubstrateProfile
repeated string capabilities = 9; // Declared substrate capabilities   bool
governance_required = 10;   string recommended_action  = 11; }
```

3.2.4 violation.proto

```
syntax = "proto3"; package infinity.violation;  option go_package =
"github.com/infinity-os/cil/proto/infinity/violation";  // Violation is the
canonical error object for any invariant, law, // or governance constraint
breach detected by IIE. message Violation {   string violation_id      =
1;  // UUID v4   string timestamp         = 2;  // RFC 3339   string source
= 3;  // CL | MSL | SL | GOVERNANCE | IIE   string invariant_id      = 4;  //
INV-### or empty   string law_id            = 5;  // LAW-### or empty
string epoch_id          = 6;  // EPOCH-UUID or empty   string severity
= 7;  // FATAL | ERROR | WARNING   string message           = 8;   bytes
context           = 9;  // JSON-encoded structured context   string
ii_engine_action  = 10; // HALT | BLOCK | ESCALATE | LOG   bool
governance_submitted = 11;   string proposal_id       = 12; //
GovernanceTicket ID or empty }
```

3.2.5 governance_ticket.proto

```
syntax = "proto3"; package infinity.governance;  option go_package =
"github.com/infinity-os/cil/proto/infinity/governance";  import
"proto/infinity/epoch.proto"; import
"proto/infinity/physics/physics_threat.proto"; import
"proto/infinity/violation/violation.proto";  // GovernanceTicket is the
canonical governance envelope. // All IIE→Governance communication is wrapped
in this type. message GovernanceTicket {   string ticket_id  = 1;  // UUID v4
string timestamp  = 2;  // RFC 3339   string source     = 3;  // IIE instance
ID    TicketKind kind   = 4;   Severity severity = 5;    Subject subject   =
6;    infinity.violation.Violation  violation     = 7;
infinity.physics.PhysicsThreat physics_threat = 8;    Recommendation
recommendation  = 9;   GovernanceState governance_state = 10;    enum
TicketKind {     VIOLATION      = 0;     PHYSICS_THREAT = 1;     COMPOSITE
= 2;  // Both violation AND physics threat   }    enum Severity {     INFO
= 0;     WARNING  = 1;     ERROR    = 2;     FATAL    = 3;     CRITICAL = 4;
}    message Subject {     string epoch_id          = 1;     string
invariant_id      = 2;     string law_id            = 3;     string
substrate_profile = 4;   }    message Recommendation {     string
ii_engine_action = 1;  // IIE's recommended action     string notes
= 2;  // Free-text reasoning   }    message GovernanceState {     Status
status             = 1;     Decision decision           = 2;     string
decision_timestamp = 3;  // RFC 3339; empty until DECIDED     string
decided_by         = 4;  // Governance principal ID      enum Status
{       OPEN         = 0;       UNDER_REVIEW = 1;       DECIDED      = 2;
CLOSED       = 3;     }      enum Decision {       NONE             = 0;
ACCEPT           = 1;       ROLLBACK         = 2;       WAIVE            = 3;
AMEND_LAW        = 4;       CHANGE_SUBSTRATE = 5;     }   } }
```

3.2.6 governance_service.proto

```
syntax = "proto3"; package infinity.governance;  option go_package =
"github.com/infinity-os/cil/proto/infinity/governance";  import
"proto/infinity/governance/governance_ticket.proto";  // GovernanceService is
the RPC boundary between IIE and Governance. // ALL IIE↔Governance
communication MUST use this service definition. service GovernanceService {
```

```
// SubmitTicket — IIE → Governance   // Called when IIE raises a violation or
physics threat requiring governance.   // Governance stores the ticket as
OPEN and returns a ticket_id.   rpc SubmitTicket (GovernanceTicket) returns
(SubmitTicketResponse);    // StreamTickets — Governance → IIE (server-
streaming)   // IIE subscribes to receive all relevant ticket updates and
decisions in   // real time. IIE MUST maintain a persistent subscription at
all times.   rpc StreamTickets (StreamTicketsRequest) returns (stream
GovernanceTicket);    // AcknowledgeDecision — IIE → Governance   // Called
after IIE has successfully applied a governance decision.   // Governance
marks the ticket CLOSED on receipt.   rpc AcknowledgeDecision
(AcknowledgeDecisionRequest)       returns (AcknowledgeDecisionResponse); }
message SubmitTicketResponse {   string ticket_id = 1;   bool   accepted  =
2;   string message   = 3;  // Rejection reason if accepted=false }  message
StreamTicketsRequest {   repeated string ticket_ids  = 1; // Empty =
subscribe to all   repeated string kinds       = 2; // Filter by TicketKind;
empty = all   repeated string severities  = 3; // Filter by Severity; empty =
all }  message AcknowledgeDecisionRequest {   string ticket_id = 1;   string
iie_id    = 2;  // IIE instance acknowledging the decision }  message
AcknowledgeDecisionResponse {   bool   applied = 1;   string message = 2; }
```

3.3 Cap'n Proto

The following Cap'n Proto schema provides the zero-copy, low-latency alternative serialization for all four core types. Use for DTD hot-path telemetry processing and high-frequency MSL→IIE metric streams. The file ID `@0xa1a1a1a1a1a1a1a1` is the canonical identifier for the CIL schema module.

```
@0xa1a1a1a1a1a1a1a1;  using Cxx = import "/capnp/c++.capnp";
$Cxx.namespace("infinity::cil");  # ──────────────────────────────────────── #
Epoch # ──────────────────────────────── struct Epoch {   epochId
@0  :Text;          # UUID v4   parentEpoch   @1  :Text;          # Empty for
genesis   timestamp     @2  :Text;          # RFC 3339   type
@3  :EpochType;   intent        @4  :Text;   input         @5  :Data;
# JSON-encoded CL input   preStateHash  @6  :Text;          # SHA-256 hex
postStateHash @7  :Text;          # SHA-256 hex; empty until COMMITTED   diff
@8  :Data;          # JSON-encoded deterministic delta   governance
@9  :GovernanceCtx;   audit         @10 :AuditInfo;    enum EpochType
{     create     @0;     mutate     @1;     rollback   @2;     governance @3;
checkpoint @4;   } }  struct GovernanceCtx {   proposalId  @0 :Text;
lawContext  @1 :List(Text); }  struct AuditInfo {   signature   @0 :Text;
validatorId @1 :Text; }  # ──────────────────────────────────────── # Invariant
# ──────────────────────────────── struct Invariant {   invariantId
@0 :Text;    # INV-### format   level           @1 :Level;   description
@2 :Text;   scope           @3 :List(Text);   checkType       @4 :CheckType;
logicRef        @5 :Text;    # WASM URI | sha256 ref | embedded expr
severity        @6 :Severity;   onViolation     @7 :Text;    # HALT | BLOCK |
ESCALATE | LOG   governanceRoute @8 :Bool;    enum Level {     constitutional
@0;     statutory      @1;     policy         @2;   }    enum CheckType {
pre     @0;     post    @1;     runtime @2;     all     @3;   }    enum
Severity {     fatal   @0;     error   @1;     warning @2;   } }  #
──────────────────────────────────────── # PhysicsThreat #
──────────────────────────────────────── struct PhysicsThreat {   threatId
@0  :Text;   timestamp          @1  :Text;   substrate          @2  :Text;
# CLASSICAL | QUANTUM | LATTICE | HYBRID   category          @3  :Text;   #
See Section 2.3.1   severity          @4  :Text;   # INFO | WARNING |
CRITICAL   metricsRaw         @5  :Data;   # JSON substrate-specific
telemetry   metricsNormalized  @6  :Data;   # JSON substrate-agnostic values
```

```
substrateProfile   @7  :Text;   # SAI-PROFILE-ID   capabilities
@8  :List(Text);   governanceRequired @9  :Bool;   recommendedAction
@10 :Text; }  # ──────────────────────────────────────── # Violation #
──────────────────────────────────── struct Violation {   violationId
@0  :Text;   timestamp           @1  :Text;   source              @2  :Text;
# CL | MSL | SL | GOVERNANCE | IIE   invariantId         @3  :Text;   # INV-
### or empty   lawId               @4  :Text;   # LAW-### or empty   epochId
@5  :Text;   # EPOCH-UUID or empty   severity            @6  :Text;   # FATAL
| ERROR | WARNING   message             @7  :Text;   context
@8  :Data;   # JSON structured context   iiEngineAction      @9  :Text;   #
HALT | BLOCK | ESCALATE | LOG   governanceSubmitted @10 :Bool;   proposalId
@11 :Text;   # GovernanceTicket ID or empty }
```

---

4. Language Type Definitions

The following type definitions are provided for Rust and Go. Both sets are normative — implementations in these languages MUST use these exact type signatures for all inter-component interface contracts. The `serde` rename attributes in Rust and the JSON struct tags in Go are normative and MUST NOT be modified.

4.1 Rust

Complete Rust type definitions. Requires `serde` and `serde_json` in `Cargo.toml`. The `r#type` raw identifier is required because `type` is a reserved keyword in Rust.

```
use serde::{Serialize, Deserialize};  //
──────────────────────────────────────── // Epoch Types //
────────────────────────────────────────  #[derive(Serialize, Deserialize,
Clone, Debug, PartialEq)] #[serde(rename_all = "SCREAMING_SNAKE_CASE")] pub
enum EpochType {     Create,     Mutate,     Rollback,     Governance,
Checkpoint, }  #[derive(Serialize, Deserialize, Clone, Debug)] pub struct
GovernanceContext {     pub proposal_id: Option<String>,     pub law_context:
Vec<String>, }  #[derive(Serialize, Deserialize, Clone, Debug)] pub struct
AuditInfo {     pub signature: String,     pub validator_id: String, }
#[derive(Serialize, Deserialize, Clone, Debug)] pub struct Epoch {     pub
epoch_id: String,     #[serde(skip_serializing_if = "Option::is_none")]
pub parent_epoch: Option<String>,     pub timestamp: String,
#[serde(rename = "type")]     pub r#type: EpochType,
#[serde(skip_serializing_if = "Option::is_none")]     pub intent:
Option<String>,     #[serde(skip_serializing_if = "Option::is_none")]     pub
input: Option<serde_json::Value>,     pub pre_state_hash: String,     pub
post_state_hash: Option<String>,     #[serde(skip_serializing_if =
"Option::is_none")]     pub diff: Option<serde_json::Value>,     pub
governance_context: GovernanceContext,     pub audit: AuditInfo, }  //
──────────────────────────────────────── // Invariant Types //
────────────────────────────────────────  #[derive(Serialize, Deserialize,
Clone, Debug, PartialEq)] #[serde(rename_all = "SCREAMING_SNAKE_CASE")] pub
enum InvariantLevel {     Constitutional,     Statutory,     Policy, }
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq)] #[serde(rename_all
```

```
= "SCREAMING_SNAKE_CASE")] pub enum CheckType {     Pre,     Post,
Runtime,     All, }  #[derive(Serialize, Deserialize, Clone, Debug,
PartialEq)] #[serde(rename_all = "SCREAMING_SNAKE_CASE")] pub enum Severity {
Fatal,     Error,     Warning, }  #[derive(Serialize, Deserialize, Clone,
Debug, PartialEq)] #[serde(rename_all = "SCREAMING_SNAKE_CASE")] pub enum
ViolationAction {     Halt,     Block,     Escalate,     Log, }
#[derive(Serialize, Deserialize, Clone, Debug)] pub struct Check
{     #[serde(rename = "type")]     pub r#type: CheckType,     pub logic:
String, }  #[derive(Serialize, Deserialize, Clone, Debug)] pub struct
OnViolation {     pub action: ViolationAction,     pub governance_route:
bool, }  #[derive(Serialize, Deserialize, Clone, Debug)] pub struct Invariant
{     pub invariant_id: String,     pub level: InvariantLevel,     pub
description: String,     pub scope: Vec<String>,     pub check: Check,
pub severity: Severity,     pub on_violation: OnViolation, }  //
──────────────────────────────────────── // PhysicsThreat Types //
────────────────────────────────────────  #[derive(Serialize, Deserialize,
Clone, Debug)] pub struct PhysicsMetrics {     pub raw: serde_json::Value,
pub normalized: serde_json::Value, }  #[derive(Serialize, Deserialize, Clone,
Debug)] pub struct MslContext {     pub substrate_profile: String,     pub
capabilities: Vec<String>, }  #[derive(Serialize, Deserialize, Clone, Debug)]
pub struct PhysicsThreat {     pub threat_id: String,     pub timestamp:
String,     pub substrate: String,     pub category: String,     pub
severity: String,     pub metrics: PhysicsMetrics,     pub msl_context:
MslContext,     pub governance_required: bool,     pub recommended_action:
String, }  // ──────────────────────────────────────── // Violation Types //
────────────────────────────────────────  #[derive(Serialize, Deserialize,
Clone, Debug)] pub struct ViolationDetails {     pub message: String,
#[serde(skip_serializing_if = "Option::is_none")]     pub context:
Option<serde_json::Value>, }  #[derive(Serialize, Deserialize, Clone, Debug)]
pub struct GovernanceEvent {     pub submitted: bool,
#[serde(skip_serializing_if = "Option::is_none")]     pub proposal_id:
Option<String>, }  #[derive(Serialize, Deserialize, Clone, Debug)] pub struct
Violation {     pub violation_id: String,     pub timestamp: String,     pub
source: String,     #[serde(skip_serializing_if = "Option::is_none")]     pub
invariant_id: Option<String>,     #[serde(skip_serializing_if =
"Option::is_none")]     pub law_id: Option<String>,
#[serde(skip_serializing_if = "Option::is_none")]     pub epoch_id:
Option<String>,     pub severity: Severity,     pub details:
ViolationDetails,     pub ii_engine_action: ViolationAction,     pub
governance_event: GovernanceEvent, }
```

4.2 Go

Complete Go type definitions. Uses `encoding/json` standard library struct tags. `time.Time` MUST be serialized in RFC 3339 format. The `omitempty` tag is used only on optional fields.

```
package infinity  import "time"  // ──────────────────────────────────────── //
Epoch Types // ────────────────────────────────────────  type EpochType string
const (     EpochCreate     EpochType = "CREATE"     EpochMutate
EpochType = "MUTATE"     EpochRollback   EpochType = "ROLLBACK"
EpochGovernance EpochType = "GOVERNANCE"     EpochCheckpoint EpochType =
"CHECKPOINT" )  type GovernanceContext struct {     ProposalID string
`json:"proposal_id"`     LawContext  []string `json:"law_context"` }  type
AuditInfo struct {     Signature   string `json:"signature"`     ValidatorID
string `json:"validator_id"` }  type Epoch struct {     EpochID       string
`json:"epoch_id"`     ParentEpoch   *string
```

```
`json:"parent_epoch,omitempty"`     Timestamp     time.Time
`json:"timestamp"`     Type          EpochType        `json:"type"`
Intent        *string          `json:"intent,omitempty"`     Input
map[string]any   `json:"input,omitempty"`     PreStateHash  string
`json:"pre_state_hash"`     PostStateHash *string
`json:"post_state_hash,omitempty"`     Diff          map[string]any
`json:"diff,omitempty"`     GovContext    GovernanceContext
`json:"governance_context"`     Audit         AuditInfo        `json:"audit"`
}  // ──────────────────────────────────── // Invariant Types //
──────────────────────────────────  type InvariantLevel  string type
CheckType       string type Severity        string type ViolationAction
string  const (     LevelConstitutional InvariantLevel  = "CONSTITUTIONAL"
LevelStatutory      InvariantLevel  = "STATUTORY"     LevelPolicy
InvariantLevel  = "POLICY"      CheckPre     CheckType = "PRE"     CheckPost
CheckType = "POST"     CheckRuntime CheckType = "RUNTIME"     CheckAll
CheckType = "ALL"      SeverityFatal   Severity = "FATAL"     SeverityError
Severity = "ERROR"     SeverityWarning Severity = "WARNING"      ActionHalt
ViolationAction = "HALT"     ActionBlock     ViolationAction = "BLOCK"
ActionEscalate  ViolationAction = "ESCALATE"     ActionLog
ViolationAction = "LOG" )  type Check struct {     Type  CheckType
`json:"type"`     Logic string    `json:"logic"` }  type OnViolation struct {
Action          ViolationAction `json:"action"`     GovernanceRoute bool
`json:"governance_route"` }  type Invariant struct {     InvariantID string
`json:"invariant_id"`     Level       InvariantLevel `json:"level"`
Description string         `json:"description"`     Scope       []string
`json:"scope"`     Check       Check          `json:"check"`     Severity
Severity       `json:"severity"`     OnViolation OnViolation
`json:"on_violation"` }  // ──────────────────────────────────────── //
PhysicsThreat Types // ──────────────────────────────────────  type
PhysicsMetrics struct {     Raw        map[string]any `json:"raw"`
Normalized map[string]any `json:"normalized"` }  type MslContext struct {
SubstrateProfile string   `json:"substrate_profile"`     Capabilities
[]string `json:"capabilities"` }  type PhysicsThreat struct {     ThreatID
string         `json:"threat_id"`     Timestamp          time.Time
`json:"timestamp"`     Substrate          string         `json:"substrate"`
Category           string         `json:"category"`     Severity
string         `json:"severity"`     Metrics            PhysicsMetrics
`json:"metrics"`     MslContext         MslContext     `json:"msl_context"`
GovernanceRequired bool           `json:"governance_required"`
RecommendedAction  string         `json:"recommended_action"` }  //
──────────────────────────────────── // Violation Types //
──────────────────────────────────  type ViolationDetails struct
{     Message string         `json:"message"`     Context map[string]any
`json:"context,omitempty"` }  type GovernanceEvent struct {     Submitted
bool    `json:"submitted"`     ProposalID *string
`json:"proposal_id,omitempty"` }  type Violation struct {     ViolationID
string           `json:"violation_id"`     Timestamp      time.Time
`json:"timestamp"`     Source         string           `json:"source"`
InvariantID    *string          `json:"invariant_id,omitempty"`     LawID
*string          `json:"law_id,omitempty"`     EpochID        *string
`json:"epoch_id,omitempty"`     Severity       Severity
`json:"severity"`     Details        ViolationDetails `json:"details"`
IIEngineAction ViolationAction  `json:"ii_engine_action"`     GovEvent
GovernanceEvent  `json:"governance_event"` }
```

5. MSL Contract — CL↔SL RPC Interface

The **MSL** (Middleware Substrate Layer) is a strict RPC boundary with ZERO cognition and ZERO governance capability. ALL CL↔SL communication MUST be routed exclusively through the MSL interface defined in this section. Direct CL-to-SL or SL-to-CL calls constitute a C-02 conformance violation.

> ⚠️ **BOUNDARY ENFORCEMENT**
>
> MSL MUST NOT modify epoch workload semantics. It translates between CL and SL representations but MUST NOT interpret, modify, or conditionally process workload payloads. Any MSL logic that branches on CL cognitive state or SL physical substrate identity (beyond normalization) is a conformance violation.

5.1 MSL Data Structures

5.1.1 SubstrateProfile

The **SubstrateProfile** is the declarative capability advertisement of an SL instance. CL uses this to determine what workloads can be dispatched to a given substrate. The MSL MUST return an up-to-date SubstrateProfile on every `DescribeSubstrate` call.

```
{   "profile_id": "SAI-PROFILE-ID",   "substrate":  "CLASSICAL | QUANTUM | LATTICE | HYBRID",   "capabilities": ["CAP-FOO", "CAP-BAR", "CAP-ENTANGLE"], "constraints": {     "max_parallelism":    1024,     "coherence_window_ns": 5000,     "max_qubit_count":    512,     "clock_precision_ns": 10   } }
```

5.1.2 EpochWorkload

The **EpochWorkload** is the ONLY mechanism by which CL is permitted to request substrate execution from SL. All substrate-bound work MUST be expressed as an EpochWorkload. The `determinism_contract` field defines the hash-based verification that MSL enforces upon receiving `EpochWorkloadResult`.

```
{   "epoch_id":    "EPOCH-UUID",   "workload_id": "WL-UUID",   "profile_id": "SAI-PROFILE-ID",   "payload": {     "encoded_program": "<bytes/base64>", "inputs": { "any": "substrate-specific inputs" }   },
```

```
"determinism_contract": {     "expected_hash": "SHA256",     "timeout_ms":
5000   } }
```

5.1.3 EpochWorkloadResult

The **EpochWorkloadResult** is the response from SL execution, normalized by MSL. MSL MUST verify `output_hash` against `determinism_contract.expected_hash` before returning to CL. If hashes do not match, MSL MUST return status=FAILED and MUST NOT deliver the output to CL.

```
{   "epoch_id":    "EPOCH-UUID",   "workload_id": "WL-UUID",   "status":
"OK | FAILED | TIMEOUT",   "output":      { "any": "substrate-specific
outputs" },   "output_hash": "SHA256",   "metrics": {     "duration_ms":
123,     "resource_usage": { "cpu_pct": 42, "qpu_shots": 1024 }   } }
```

5.1.4 PhysicsEvent (Raw SL Emission)

The **PhysicsEvent** is the raw, untyped anomaly report emitted by an SL instance. MSL MUST intercept all PhysicsEvents and normalize them into typed **PhysicsThreat** objects before any further propagation. PhysicsEvents MUST NEVER reach IIE directly.

```
{   "event_id":   "PE-UUID",   "timestamp":  "RFC3339",   "profile_id": "SAI-
PROFILE-ID",   "substrate":  "CLASSICAL | QUANTUM | LATTICE | HYBRID",
"raw_metrics": { "any": "untyped substrate telemetry" } }
```

5.2 MSL Operations

Operation	Direction	Input	Output	Description
MSL.DescribeSubstrate()	CL → MSL → SL	(none)	SubstrateProfile	Returns the current SubstrateProfile. CL MUST call this before constructing any EpochWorkload to verify substrate capabilities match workload requirements.
MSL.ExecuteEpochWorkload(EpochWorkload)	CL → MSL → SL	EpochWorkload	EpochWorkloadResult	Executes the substrate-bound portion of an epoch.

Operation	Direction	Input	Output	Description
				MSL enforces the determinism contract on the result before returning to CL.
MSL.StreamMetrics(Subscription)	SL → MSL → IIE	Subscription config	stream PhysicsEvent	Optional continuous telemetry stream. MSL normalizes each event to PhysicsThreat before forwarding to IIE DTD. SHOULD be enabled for QUANTUM and LATTICE substrates.
MSL.ReportPhysicsEvent(PhysicsEvent)	SL → MSL → IIE	PhysicsEvent	PhysicsThreat	Point-in-time anomaly report. MSL normalizes the PhysicsEvent to a typed PhysicsThreat and calls IIE.ReportPhysicsThreat(). Returns the normalized PhysicsThreat for SL audit logging.

5.3 MSL Hard Constraints

Constraint ID	Rule	Violation Result
MSL-C1	MSL MUST NOT modify the epoch workload	C-02 conformance violation; FATAL Violation

Constraint ID	Rule	Violation Result
	semantics — it translates, never interprets	
MSL-C2	MSL MUST normalize ALL SL anomalies into typed PhysicsThreat before passing to IIE	C-13 conformance violation; raw event delivery to IIE is non-conformant
MSL-C3	MSL MUST enforce the determinism contract: if output_hash ≠ expected_hash → status=FAILED	TIMING_ANOMALY PhysicsThreat emitted; epoch rejected
MSL-C4	MSL MUST NOT cache or accumulate CL state across EpochWorkload calls	C-05 conformance violation
MSL-C5	MSL MUST NOT communicate with GovernanceService — all governance routing is via IIE	Architecture violation; C-05
MSL-C6	MSL MUST propagate timeout_ms from EpochWorkload.determinism_contract to SL execution	TIMEOUT status on result if SL exceeds timeout_ms

---

6. IIE Runtime API

The **IIE** (Invariant Inference Engine) Runtime API is the authoritative programmatic interface for all epoch lifecycle management, invariant enforcement, threat reporting, and governance bridge operations. This API is language-agnostic and MUST be bound to a concrete RPC layer (gRPC, internal IPC, or WASM ABI) by each implementation. All operations return `Result<T, Violation>`.

6.1 Epoch Lifecycle

```
// ProposeEpoch — Pre-execution validation gate // CL calls this before any SL execution begins. // IIE evaluates all PRE-type invariants for the epoch's scope and type. // Returns EpochAuthorization on success; Violation on any
```

```
PEV failure. Result<EpochAuthorization, Violation>  IIE.ProposeEpoch(Epoch
epoch)  // CommitEpoch — Post-execution commitment gate // CL calls this
after SL execution completes successfully. // IIE evaluates all POST-type
invariants, writes the audit entry, // and sets post_state_hash. Returns
EpochCommit on success; // Violation on any PXV failure (epoch transitions to
ROLLED_BACK). Result<EpochCommit, Violation>         IIE.CommitEpoch(Epoch
epoch)
```

6.1.1 EpochAuthorization

```
{   "epoch_id":            "EPOCH-UUID",   "authorized":          true,
"iie_id":            "IIE-INSTANCE-ID",   "timestamp":          "RFC3339",
"constraints_applied": ["INV-001", "INV-004", "LAW-010"] }
```

6.1.2 EpochCommit

```
{   "epoch_id":        "EPOCH-UUID",   "committed":       true,
"post_state_hash": "SHA256",   "audit_entry_id":  "AUDIT-UUID",
"timestamp":       "RFC3339" }
```

Field	Type	Description
`epoch_id`	string	The epoch being authorized or committed
`authorized` / `committed`	bool	True if IIE approved the operation. If false, the accompanying Violation explains the rejection.
`iie_id`	string	The IIE instance that processed the request. Written to audit ledger.
`constraints_applied`	string[]	List of INV-### and LAW-### that IIE evaluated during this call. Audit evidence.
`post_state_hash`	string	(CommitEpoch only) The SHA-256 hash of CL state after commit. IIE writes this to the epoch record.
`audit_entry_id`	string	(CommitEpoch only) The ID of the audit ledger entry written for this epoch transition.

6.2 Invariant Management

```
// ListInvariants — Return all active invariants for a given scope // Scope:
"CL" | "MSL" | "SL" | "GOVERNANCE" | "" (all scopes) List<Invariant>
IIE.ListInvariants(string scope)  // RegisterInvariant — Register a new
invariant in the IIE registry // L0 (CONSTITUTIONAL) and L1 (STATUTORY)
```

```
invariants REQUIRE // a valid GovernanceTicket with decision=AMEND_LAW. // L2 (POLICY) invariants require operator authorization. Result<(), Violation> IIE.RegisterInvariant(Invariant inv)  // DisableInvariant — Disable an active invariant // ALL levels require governance authorization (AMEND_LAW or WAIVE decision). // Disabled invariants are retained in registry with status=DISABLED; // they MUST NOT be deleted (audit trail integrity). Result<(), Violation>         IIE.DisableInvariant(string invariantId)
```

> ⚠️ **INVARIANT REGISTRY INTEGRITY**
>
> Disabled invariants MUST be retained in the IIE registry with
>
> status=DISABLED
>
> . Deletion of invariant records is non-conformant. The registry is an append-only
>
> audit artifact. To re-enable a disabled invariant, a new
>
> RegisterInvariant
>
> call is required with governance authorization.

6.3 Threat & Drift Reporting

```
// ReportPhysicsThreat — Receive a normalized PhysicsThreat from MSL // IIE DTD classifies the threat, determines severity escalation, and: //   INFO: logs to audit ledger //   WARNING:  logs + notifies governance observer // CRITICAL: suspends all EXECUTING epochs on affected substrate, // blocks new ProposeEpoch, constructs GovernanceTicket // Returns () on success; Violation if IIE cannot classify or process. Result<(), Violation> IIE.ReportPhysicsThreat(PhysicsThreat threat)
```

6.4 Governance Bridge

```
// RaiseViolation — Raise a violation into governance via GB // IIE constructs a GovernanceTicket wrapping the Violation // and calls GovernanceService.SubmitTicket. // Returns the submitted GovernanceTicket on success. Result<GovernanceTicket, Violation>  IIE.RaiseViolation(Violation vio)  // RequestException — Request a governance waiver for a violation // Used when CL needs to continue despite a non-FATAL violation. // Governance may respond with WAIVE, ACCEPT, or ROLLBACK decision. // IIE MUST NOT resume execution until the decision is received. Result<GovernanceTicket, Violation> IIE.RequestException(Violation vio)
```

6.5 Error Semantics

All IIE API operations return `Result<T, Violation>`. The **Violation** in the error case MUST contain the following:

Field	Required Content
`invariant_id` or `law_id`	The specific invariant or law that caused the rejection. At least one MUST be populated.
`epoch_id`	The epoch in context, if the error is epoch-bound. Null for system-level errors.
`ii_engine_action`	The action IIE took: HALT, BLOCK, ESCALATE, or LOG. Callers MUST respect this action and MUST NOT retry without governance resolution if action is HALT or BLOCK.
`governance_event.submitted`	Whether IIE submitted a GovernanceTicket. If true, callers MUST wait for governance resolution before retrying.

> ⚠️ **RETRY PROHIBITION**
>
> Callers MUST NOT retry a rejected ProposeEpoch or CommitEpoch call if the Violation contains
>
> ii_engine_action = HALT
>
> or
>
> ii_engine_action = BLOCK
>
> . Any retry under HALT or BLOCK constitutes a C-09 conformance violation (ad-hoc CL mutation outside epochs).

---

7. Governance API — IIE↔Governance RPC

7.1 GovernanceService Definition

The **GovernanceService** is the exclusive RPC boundary between the IIE and the Governance subsystem. All three RPCs defined in `governance_service.proto` (Section 3.2.6) are normative and MUST be implemented by any conformant GovernanceService deployment.

7.1.1 SubmitTicket — IIE → Governance

Triggered when IIE detects: an invariant violation requiring governance review, a CRITICAL or WARNING PhysicsThreat, or a composite event combining both. The GovernanceService MUST store the ticket in OPEN state and return the `ticket_id` synchronously.

Step	Actor	Action
1	IIE (GB subsystem)	Constructs GovernanceTicket with kind, severity, subject, and recommendation populated
2	IIE → GovernanceService	Calls `SubmitTicket(ticket)`
3	GovernanceService	Validates ticket schema; stores with status=OPEN
4	GovernanceService → IIE	Returns `SubmitTicketResponse{ticket_id, accepted=true}`
5	IIE	Records `governance_event.proposal_id = ticket_id` in the originating Violation

7.1.2 StreamTickets — Governance → IIE (Server-Streaming)

IIE MUST maintain a persistent `StreamTickets` subscription to receive all governance decisions, law amendments, operator overrides, and ticket updates in real time. Loss of the streaming connection MUST be treated as a WARNING event and reconnection attempted within implementation-defined bounds.

Step	Actor	Action
1	IIE	Calls `StreamTickets({})` at startup — subscribes to all ticket events
2	GovernanceService	Streams GovernanceTicket updates as they occur (OPEN→UNDER_REVIEW, UNDER_REVIEW→DECIDED)
3	IIE	On receipt of a DECIDED ticket, IIE applies the governance decision immediately
4	IIE	After applying decision, calls `AcknowledgeDecision`

7.1.3 AcknowledgeDecision — IIE → Governance

After IIE has successfully applied a governance decision (performed a rollback, halted a component, updated the invariant registry, migrated substrate, or entered safe-mode), IIE MUST call `AcknowledgeDecision`. GovernanceService marks the ticket CLOSED on receipt. Failure to acknowledge within the implementation-defined SLA is a WARNING-level governance event.

7.2 GovernanceTicket State Machine

From State	To State	Trigger	Notes
OPEN	UNDER_REVIEW	Governance process assigns reviewer and begins evaluation	IIE MUST continue holding affected execution suspended during review
UNDER_REVIEW	DECIDED	Governance principal issues a decision (ACCEPT, ROLLBACK, WAIVE, AMEND_LAW, CHANGE_SUBSTRATE)	decision_timestamp and decided_by MUST be populated
DECIDED	CLOSED	IIE calls	Terminal state —

From State	To State	Trigger	Notes
		AcknowledgeDecision confirming decision has been applied	no further transitions permitted
DECIDED	UNDER_REVIEW	Decision appealed by Operator or superseded by higher-level governance authority	Only permitted within implementation-defined appeal window

7.3 Decision → IIE Action Mapping

Governance Decision	IIE Action Required	Epoch Effect
`ACCEPT`	Resume normal execution; log acceptance in audit ledger	Suspended epochs may be resumed; PROPOSED epochs may proceed to ProposeEpoch
`ROLLBACK`	Roll back the affected epoch to the last CHECKPOINT; log rollback in audit ledger	Epoch transitions to ROLLED_BACK; CL state reverts to checkpoint hash
`WAIVE`	Mark the specific invariant violation as waived for this occurrence; log waiver with proposal_id	Epoch may proceed past the waived invariant; waiver is non-reusable (per-epoch)
`AMEND_LAW`	Update invariant registry with new law definition per the ticket payload; reload affected invariants	Future epochs evaluated against amended invariant set; in-flight epochs evaluated against prior set
`CHANGE_SUBSTRATE`	Isolate the affected substrate profile; migrate pending workloads to the alternate profile specified in ticket	In-flight epochs on affected substrate: ROLLED_BACK; new epochs dispatched to alternate profile
`NONE`	No action; ticket is informational only	No epoch effect; audit ledger entry written

---

8. State Machines

All state machines below are normative. Implementations MUST enforce these state transition rules and MUST reject any transition not defined here. Illegal state transitions MUST be logged as FATAL Violations.

8.1 Epoch Lifecycle State Machine

States: **PROPOSED** → **AUTHORIZED** → **EXECUTING** → **COMPLETED** → {**COMMITTED** | **REJECTED** | **ROLLED_BACK**}

```
PROPOSED   ├─ [PEV: all PRE invariants pass]        ──────────────────────→
AUTHORIZED   └─ [PEV: any PRE invariant fails]        ──────────────────────→
REJECTED      [governance veto during proposal]      ──────────────────────→
REJECTED      (+ GovernanceTicket if governance_route=true)  AUTHORIZED   └─
[MSL.ExecuteEpochWorkload dispatched]  ──────────────────────→ EXECUTING
EXECUTING   ├─ [SL execution completes: status=OK]    ──────────────────────→
COMPLETED   ├─ [SL execution: status=FAILED]          ──────────────────────→
REJECTED   ├─ [SL execution: status=TIMEOUT]         ──────────────────────→
REJECTED   └─ [RG detects boundary violation]        ──────────────────────→
REJECTED      (+ GovernanceTicket)  COMPLETED   ├─ [PXV: all POST invariants
pass       + audit entry written]                  ──────────────────────→
COMMITTED   └─ [PXV: any POST invariant fails]        ──────────────────────→
ROLLED_BACK      [governance veto post-execution]       ──────────────────────→
ROLLED_BACK      (+ GovernanceTicket)  COMMITTED    — Terminal. Immutable.
post_state_hash written. REJECTED     — Terminal. Epoch discarded. No state
change to CL. ROLLED_BACK  — Terminal. CL state reverted to pre_state_hash.
GovernanceTicket submitted.
```

Transition	Guard Condition	IIE Subsystem	Audit Entry
PROPOSED → AUTHORIZED	All PRE invariants in scope return true	PEV	YES
PROPOSED → REJECTED	Any PRE invariant returns false	PEV	YES
AUTHORIZED → EXECUTING	EpochWorkload dispatched to MSL	RG	YES
EXECUTING → COMPLETED	output_hash verified; status=OK	MSL + RG	YES
EXECUTING → REJECTED	status=FAILED | TIMEOUT | RG boundary violation	RG + MSL	YES
COMPLETED → COMMITTED	All POST invariants pass + audit entry	PXV	YES

Transition	Guard Condition	IIE Subsystem	Audit Entry
	written		
COMPLETED → ROLLED_BACK	Any POST invariant fails or governance veto	PXV + GB	YES

8.2 Violation Lifecycle State Machine

States: **DETECTED → LOGGED → ESCALATED → RESOLVED**

```
DETECTED  (IIE identifies invariant or law breach)   └─ [IIE writes Violation record to audit ledger] ──→ LOGGED  LOGGED   ├─ [severity=WARNING AND governance_route=false]  ──→ RESOLVED (LOG only)   └─ [severity != WARNING OR governance_route=true]                        ──→ ESCALATED (GovernanceTicket submitted via GB)  ESCALATED   └─ [GovernanceTicket reaches DECIDED state;       decision applied by IIE;       AcknowledgeDecision called]                  ──→ RESOLVED  RESOLVED  — Terminal. Violation record frozen. No further state changes.
```

8.3 PhysicsThreat Lifecycle State Machine

States: **OBSERVED → CLASSIFIED → ACTED → CLOSED**

```
OBSERVED  (SL emits PhysicsEvent)   └─ [MSL normalizes to PhysicsThreat; IIE.ReportPhysicsThreat called;       DTD evaluates category and severity] ──────────→ CLASSIFIED  CLASSIFIED   ├─ [severity=INFO]   │    └─ [LOG to audit ledger]              ──────────→ ACTED   │   ├─ [severity=WARNING]   │   └─ [LOG + notify governance observer   │         + throttle affected workloads]   ──────────→ ACTED   │   └─ [severity=CRITICAL]        └─ [suspend all EXECUTING epochs            on affected substrate;            block new ProposeEpoch calls;            submit GovernanceTicket(CRITICAL)]──────────→ ACTED  ACTED   └─ [substrate stabilizes;       governance signs off (ACCEPT or       CHANGE_SUBSTRATE decision applied);       IIE.AcknowledgeDecision called]        ──────────→ CLOSED  CLOSED  — Terminal.
```

8.4 GovernanceTicket Lifecycle State Machine

Defined fully in Section 7.2. Abbreviated state flow for reference:

```
OPEN → UNDER_REVIEW → DECIDED → CLOSED                        ↑           (on appeal)              UNDER_REVIEW ←─────(DECIDED, within appeal window)
```

---

9. End-to-End Implementation Flows

The following flows are normative implementation references. Engineers MUST verify their implementations produce identical component interactions and data structure states at each step.

9.1 Flow: Normal Epoch Execution (Happy Path)

This flow describes a MUTATE epoch completing successfully across all CIL layers with no invariant violations. This is the baseline conformance flow and MUST be verified by the integration test suite before any other flows are tested.

Step	Component	Interface	Data Structure	State After
1	CL	—	Constructs `Epoch{type=MUTATE, intent="update agent state", pre_state_hash=H₀, post_state_hash=null}`	Epoch: PROPOSED
2	CL → IIE	`IIE.ProposeEpoch(epoch)`	Epoch submitted to IIE PEV	PEV evaluation begins
3	IIE PEV	—	Evaluates all PRE invariants in CL scope: INV-001 ✓, INV-002 ✓, INV-003 ✓, INV-004 ✓ → EpochAuthorization{authorized=true, constraints_applied=[INV-001, INV-002, INV-003, INV-004]}	Epoch: AUTHORIZED
4	IIE → MSL	`MSL.DescribeSubstrate()`	SubstrateProfile returned; capabilities verified against EpochWorkload	SubstrateProfile confirmed

Step	Component	Interface	Data Structure	State After
			requirements	
5	IIE → MSL	`MSL.ExecuteEpochWorkload(workload)`	EpochWorkload{epoch_id, profile_id, payload, determinism_contract.expected_hash=H_1, timeout_ms=5000}	Epoch: EXECUTING
6	MSL → SL	Substrate ABI	Workload executed; metrics streamed to DTD during execution	SL executing
7	SL → MSL	MSL internal	EpochWorkloadResult{status=OK, output_hash=H_1}; MSL verifies H_1 == expected_hash ✓	COMPLETED locally
8	IIE PXV	—	Evaluates all POST invariants; output_hash verified; determinism_contract satisfied; audit entry written	Epoch: COMPLETED
9	CL → IIE	`IIE.CommitEpoch(epoch)`	EpochCommit{committed=true, post_state_hash=H_1, audit_entry_id=AE-UUID}	Epoch: COMMITTED
10	IIE Audit	Ledger append	Full epoch transition log (PROPOSED→COMMITTED) written; post_state_ha	Audit ledger updated

Step	Component	Interface	Data Structure	State After
			sh=H_1 written to epoch record	

9.2 Flow: Quantum Decoherence → Composite GovernanceTicket

This flow describes the canonical CRITICAL error path: a QPU coherence collapse during an in-flight epoch, triggering substrate suspension, governance escalation, and substrate migration. This flow MUST be covered by the conformance test suite.

Step	Component	Interface	Data Structure	State After
1	SL (QPU)	SL hardware	QPU emits `PhysicsEvent{substrate=QUANTUM, raw_metrics={coherence_time_ns: 42}}`	PhysicsThreat: OBSERVED
2	MSL	Internal normalization	PhysicsThreat {category=COHERENCE_COLLAPSE, severity=CRITICAL, governance_required=true, metrics.normalized={coherence_ratio: 0.008}}	PhysicsThreat: CLASSIFIED
3	MSL → IIE	`IIE.ReportPhysicsThreat(threat)`	Normalized PhysicsThreat delivered to IIE DTD	DTD receiving
4	IIE DTD	—	DTD classifies: CRITICAL; INV-001 violation detected (determinism guarantee cannot be	Violation: DETECTED → LOGGED

Step	Component	Interface	Data Structure	State After
			upheld). Violation{severity=FATAL, invariant_id="INV-001", source=IIE, ii_engine_action=HALT} constructed.	
5	IIE GB	—	Constructs GovernanceTicket{kind=COMPOSITE, severity=CRITICAL, subject.epoch_id=current_epoch, violation=..., physics_threat=..., recommendation.ii_engine_action=CHANGE_SUBSTRATE}	Ticket constructed
6	IIE → GovernanceService	`GovernanceService.SubmitTicket(ticket)`	SubmitTicketResponse{ticket_id=GT-UUID, accepted=true}	GovernanceTicket: OPEN
7	IIE	—	Suspends all EXECUTING epochs on QUANTUM substrate; blocks ProposeEpoch for any CL dispatching to QUANTUM substrate; applies HALT action	Epoch: EXECUTING (suspended); CL blocked
8	GovernanceService	Internal	Governance reviews ticket; severity=CRIT	GovernanceTicket: DECIDED

Step	Component	Interface	Data Structure	State After
			ICAL → decision=CHANGE_SUBSTRATE; GovernanceTicket.governance_state updated to {status=DECIDED, decision=CHANGE_SUBSTRATE, decided_by="GOV-PRINCIPAL-001"}	
9	GovernanceService → IIE	`StreamTickets` stream	Updated GovernanceTicket{status=DECIDED, decision=CHANGE_SUBSTRATE} delivered to IIE via persistent subscription	IIE receives decision
10	IIE	—	Applies CHANGE_SUBSTRATE decision: isolates QUANTUM substrate profile; rolls back current epoch (COMPLETED→ROLLED_BACK); updates SubstrateProfile routing to LATTICE alternate profile; re-enables ProposeEpoch	Epoch: ROLLED_BACK; PhysicsThreat: ACTED
11	IIE →	`GovernanceSer`	Acknowledge	GovernanceTi

Step	Component	Interface	Data Structure	State After
	GovernanceService	`vice.AcknowledgeDecision({GT-UUID, iie_id})`	DecisionResponse{applied=true}	cket: CLOSED; PhysicsThreat: CLOSED
12	IIE Audit	Ledger append	Full event trace written: PhysicsThreat classification, Violation creation, GovernanceTicket lifecycle, ROLLBACK action, substrate migration — all append-only ledger entries	Audit ledger complete

9.3 Flow: Invariant Violation → Governance Escalation

This flow describes a PEV-detected FATAL violation during ProposeEpoch: CL attempts to bypass a governance constraint, triggering immediate halt and governance review.

Step	Component	Interface	Data Structure / Action	State After
1	CL → IIE	`IIE.ProposeEpoch(epoch)`	Epoch{type=MUTATE, governance_context.proposal_id=null} submitted; law_context indicates a governance-required law applies	PEV evaluation begins
2	IIE PEV	—	PEV evaluates INV-004 (governance constraint):	Violation: DETECTED → LOGGED

Step	Component	Interface	Data Structure / Action	State After
			governance_context.proposal_id=null but law requires active proposal → INV-004 FAILS. Violation{severity=FATAL, invariant_id="INV-004", ii_engine_action=HALT, source=CL} constructed.	
3	IIE GB	`IIE.RaiseViolation(violation)`	GovernanceTicket{kind=VIOLATION, severity=FATAL, subject.epoch_id=E, subject.invariant_id="INV-004"} constructed	Ticket constructed
4	IIE	—	ii_engine_action=HALT applied; epoch transitions PROPOSED→REJECTED; ProposeEpoch returns Err(violation) to CL; CL MUST NOT retry	Epoch: REJECTED; Violation: ESCALATED
5	IIE → GovernanceService	`GovernanceService.SubmitTicket(ticket)`	SubmitTicketResponse{ticket_id=GT-UUID2, accepted=true}; GovernanceTicket status=OPEN	GovernanceTicket: OPEN

Step	Component	Interface	Data Structure / Action	State After
6	GovernanceService	Internal	Governance reviews; determines epoch should not proceed; decision=ROLLBACK (belt-and-suspenders; epoch already REJECTED)	GovernanceTicket: DECIDED
7	IIE → GovernanceService	`GovernanceService.AcknowledgeDecision`	IIE confirms decision applied; GovernanceTicket→CLOSED; Violation→RESOLVED	GovernanceTicket: CLOSED; Violation: RESOLVED

---

10. Conformance Checklist

Implementations MUST satisfy all items marked MUST in this checklist to achieve CIL v1.0 conformance. Verification methods are normative — implementations MUST demonstrate compliance using the specified mechanism.

ID	Requirement	Layer	Severity	Verified By
C-01	CL, MSL, and SL implemented as distinct layers with enforced runtime boundaries	Architecture	MUST	RG integration test + architecture review
C-02	All CL↔SL traffic passes through MSL exclusively;	MSL	MUST	RG boundary monitor; network trace analysis

ID	Requirement	Layer	Severity	Verified By
	no direct CL-SL calls			
C-03	CL contains no substrate-specific logic; all substrate interaction via EpochWorkload	CL	MUST	Static analysis + PEV invariant check
C-04	SL contains no cognitive or governance logic; no Invariant or GovernanceTicket types	SL	MUST	Code review + binary dependency audit
C-05	MSL contains no cognitive or governance logic; no CL state accumulation	MSL	MUST	Code review + MSL-C4 enforcement test
C-06	All CL state changes expressed as typed Epoch objects	CL	MUST	PEV invariant check; audit ledger scan for non-epoch mutations
C-07	Epochs are deterministic: given identical pre_state_hash and input, identical post_state_hash is produced	CL	MUST	PXV hash verification; replay test suite
C-08	Epochs are idempotent from CHECKPOINT epochs: replay from checkpoint produces identical state	CL	MUST	Checkpoint replay test suite
C-09	No ad-hoc CL mutation	CL	MUST	RG runtime boundary

ID	Requirement	Layer	Severity	Verified By
	outside epoch lifecycle; no direct state writes			monitor + PEV
C-10	IIE validates epochs pre-execution via PEV subsystem; ProposeEpoch is synchronous gate	IIE	MUST	PEV unit tests; rejected epoch trace verification
C-11	IIE guards execution boundaries via RG subsystem; runtime violations produce REJECTED transitions	IIE	MUST	RG integration tests; boundary injection tests
C-12	IIE verifies post-execution state via PXV subsystem; CommitEpoch is synchronous gate	IIE	MUST	PXV unit tests; hash mismatch injection test
C-13	IIE detects and classifies PhysicsThreats via DTD subsystem for all five threat categories	IIE	MUST	DTD injection tests (one per category)
C-14	IIE integrates with GovernanceService via GB subsystem; all three RPCs functional	IIE	MUST	GB contract tests; end-to-end flow 9.2
C-15	GovernanceSe	Governance	MUST	Proto3

ID	Requirement	Layer	Severity	Verified By
	rvice implements SubmitTicket, StreamTickets, and Acknowledge Decision per Section 7			contract tests; streaming integration test
C-16	GovernanceTicket is the canonical governance envelope; all IIE→Governance communication uses this type	Governance	MUST	Schema validation against governance_ticket.json
C-17	Governance decisions applied before execution resumes after HALT or BLOCK	IIE + Governance	MUST	End-to-end flow 9.2 and 9.3; resume-before-ack injection test
C-18	Tamper-evident append-only audit ledger maintained; ledger writes are atomic	Platform	MUST	Ledger integrity test; hash-chain verification
C-19	All epoch state transitions logged with component, interface, and timestamp	IIE	MUST	Audit completeness test; state machine trace verification
C-20	All governance decisions logged with decided_by, decision_timestamp, and ticket_id	Governance	MUST	Governance audit completeness test

ID	Requirement	Layer	Severity	Verified By
C-21	All PhysicsThreat and Violation events logged in audit ledger immediately upon DETECTED	IIE	MUST	Audit completeness test; event injection trace
C-22	Audit logs are append-only; no post-write modification or deletion permitted	Platform	MUST	Write-once storage verification; modification attempt rejection test

> ⚠️ **CONFORMANCE GATE**
>
> All 22 checklist items MUST pass before a CIL+IIE implementation is declared conformant. Partial conformance is not recognized. Items C-01 through C-05 (layer boundary enforcement) are prerequisites for all other items — they MUST be verified first.

---

11. Appendix — Schema Index

All schema files referenced in this specification are listed below. Canonical schema URIs and package paths are normative. Implementations MUST use these identifiers in their schema registries and build systems.

Schema File	Canonical ID / Package	Format	Section	Defines
`epoch.json`	`https://infinity.os/schema/`	JSON Schema Draft-07	3.1.1	Epoch, GovernanceCo

Schema File	Canonical ID / Package	Format	Section	Defines
	epoch.json			ntext, AuditInfo
physics_threat.json	https://infinity.os/schema/physics_threat.json	JSON Schema Draft-07	3.1.2	PhysicsThreat, MslContext, PhysicsMetrics
violation.json	https://infinity.os/schema/violation.json	JSON Schema Draft-07	3.1.3	Violation, ViolationDetails, GovernanceEvent
epoch.proto	package infinity	proto3	3.2.1	Epoch, GovernanceContext, AuditInfo, EpochType
invariant.proto	package infinity	proto3	3.2.2	Invariant, Level, CheckType, Severity
physics_threat.proto	package infinity.physics	proto3	3.2.3	PhysicsThreat
violation.proto	package infinity.violation	proto3	3.2.4	Violation
governance_ticket.proto	package infinity.governance	proto3	3.2.5	GovernanceTicket, TicketKind, Severity, Subject, Recommendation, GovernanceState
governance_service.proto	package infinity.governance	proto3	3.2.6	GovernanceService, SubmitTicketResponse, StreamTicketsRequest, AcknowledgeDecisionRequest, Acknowledge

Schema File	Canonical ID / Package	Format	Section	Defines
				DecisionResponse
`infinity.capnp`	`@0xa1a1a1a1a1a1a1a1` / `namespace infinity::cil`	Cap'n Proto v1.0	3.3	Epoch, Invariant, PhysicsThreat, Violation, GovernanceCtx, AuditInfo

11.1 Go Module Paths

Package	Module Path	Contains
Core types	`github.com/infinity-os/cil/types`	Epoch, Invariant, PhysicsThreat, Violation (Section 4.2)
Proto — infinity	`github.com/infinity-os/cil/proto/infinity`	epoch.proto, invariant.proto generated types
Proto — physics	`github.com/infinity-os/cil/proto/infinity/physics`	physics_threat.proto generated types
Proto — violation	`github.com/infinity-os/cil/proto/infinity/violation`	violation.proto generated types
Proto — governance	`github.com/infinity-os/cil/proto/infinity/governance`	governance_ticket.proto, governance_service.proto generated types + gRPC stubs
MSL contract	`github.com/infinity-os/cil/msl`	SubstrateProfile, EpochWorkload, EpochWorkloadResult, PhysicsEvent
IIE API	`github.com/infinity-os/cil/iie`	IIE interface, EpochAuthorization, EpochCommit

11.2 Rust Crate Structure

Crate	Path	Contains
`cil-types`	`crates/cil-types`	All core types from Section 4.1; serde derives included
`cil-proto`	`crates/cil-proto`	prost-generated types from all .proto files
`cil-msl`	`crates/cil-msl`	MSL contract types and trait definitions
`cil-iie`	`crates/cil-iie`	IIE Runtime API trait definitions
`cil-governance`	`crates/cil-governance`	GovernanceService tonic-generated gRPC stubs

11.3 Normative Invariant ID Registry (Baseline)

Invariant ID	Level	Scope	Severity	Brief Description
`INV-001`	CONSTITUTIONAL	CL	FATAL	Epoch determinism: given identical S and I, output MUST be identical
`INV-002`	CONSTITUTIONAL	CL	FATAL	Epoch idempotency: replay from CHECKPOINT MUST produce identical state
`INV-003`	CONSTITUTIONAL	CL, MSL, SL	FATAL	Layer boundary integrity: CL MUST NOT contain substrate logic; SL MUST NOT contain cognitive logic
`INV-004`	CONSTITUTIONAL	CL, GOVERNANCE	FATAL	Governance constraint: ROLLBACK and GOVERNANCE

Invariant ID	Level	Scope	Severity	Brief Description
				epochs MUST have a valid governance proposal_id
INV-005	STATUTORY	CL	ERROR	Audit completeness: every epoch transition MUST produce a ledger entry within the same transaction
INV-006	STATUTORY	IIE	FATAL	Physics threat response: CRITICAL PhysicsThreat MUST trigger immediate epoch suspension on affected substrate
INV-007	POLICY	MSL	WARNING	Metric normalization freshness: normalized metrics MUST be derived from raw metrics with timestamp delta < threshold_ms